Parallel Programming HW4

113062531 楊峻豪

Implementation

I、 Algorithms

Flash Attention 的核心目標是解決傳統注意力機制中最嚴重的 bottleneck：memory bound。 標準 Attention 需要建立一個 N×N 的 score matrix $S = Q\ K^T$ ，這個矩陣不但巨大，也會導致後續 softmax、dropout、矩陣乘法都需要反覆存取 VRAM，使得延遲與能耗都大幅上升。在 paritial softmax 的部分，傳統 S 要計算 softmax 時，每個位置的值都依賴整列：

$$\mathrm{softmax}(S_i) = \frac{e^{S_i}}{\sum_j e^{S_j}}$$

Parital softmax 利用 running max m 與 running sum l 在每處理一個 tile 後即時更新 softmax 統計量，並在遇到更大的 tile 最大值時對舊的累積量以 $e\,m - n\,e\,w_m$ 進行 rescale，確保數值穩定同時避免建立整張 N×N 的 score matrix。

For vectors $x^{(1)}, x^{(2)} \in \mathbb{R}^B$, we can decompose the softmax of the concatenated $x = \left[x^{(1)}\ x^{(2)}\right] \in \mathbb{R}^{2B}$ as:

$$m(x) = m(\left[x^{(1)}\ x^{(2)}\right]) = \max(m(x^{(1)}), m(x^{(2)})), \quad f(x) = \left[e^{m(x^{(1)})-m(x)} f(x^{(1)}) \quad e^{m(x^{(2)})-m(x)} f(x^{(2)})\right],$$

$$\ell(x) = \ell(\left[x^{(1)}\ x^{(2)}\right]) = e^{m(x^{(1)})-m(x)} \ell(x^{(1)}) + e^{m(x^{(2)})-m(x)} \ell(x^{(2)}), \quad \mathrm{softmax}(x) = \frac{f(x)}{\ell(x)}.$$

II、 Data

資料處理方面，我在資料切割上把能放入快取層級(register/shared memory)

的資料盡量往裡面推，把不能承受的中間產物像完整的 S 矩陣完全避免生成。具體來說，我依序列維度 N 將 Q、K、V 切成小 tiles，例如一次處理 Br 個 query rows 與 Bc 個 key/value rows (Br 跟 Bc 如何決定下個小節會講)，並把這些 tiles 載入 shared memory。由於 Q/K/V 的 dot product 極度 memory-bound，透過 tile reuse，可以讓同一塊 K/V 在多個 query 之間重複使用，只需從 global memory 讀取一次即可，大幅減少 DRAM traffic。此外，我讓每個 thread 負責整個 1×d 的 query row，讓它持有該列 softmax 所需的全部狀態，因此 running max、running sum，以及該 tile 的 scores（例如 S_row[32]）都能放在 registers 中，不需要 thread 之間共享資訊，也不需 synchronized。

III、 Configuration

由於 shared memory 只有 48KB，而我要放進去的資料包括 Q、K、V（O 會在最後再寫回 global memory，不需要存整塊），而每個元素都是 float（4 bytes），所以我先估算每次能塞多大的 tile。假設使用的 hidden dimension 為 d ∈ {32, 64}，若我選擇 Br 個 query rows、Bc 個 key/value rows，shared memory 需要的空間大約是：

(Br * d + Bc * d + Bc * d) * 4 bytes

其中分別對應 Q tile、K tile、V tile。以 d = 64 做最壓力測試，把式子簡化成 4 * (Br + 2*Bc) * 64 bytes。在 48KB 上限下，我挑 Br = 64、Bc = 32，

代入後大小約為：4 × (64 + 64) × 64 = 32768 bytes，剛好安全落在 shared memory 內。因此 Br=64 和 Bc=32 是能兼顧容量與效能的 tile size。接著根據序列長度 N，我需要沿著 N/Br 方向切出多個 blocks 來平行處理所有 query，block 數量就是 Tr = ceil_div(N, Br)。例如 N=1024、Br=64，就會有 Tr = 16 個 blocks。同時因為我設計成「一個 thread 對應一整個 query row」，所以每個 block 內我就啟動 Br 個 threads（也就是 64 threads），讓每個 thread 專心處理自己的 1×d 向量，包括 dot product、partial softmax 的 running max / running sum 更新，完全不需要與其他 threads 溝通。

Profiling Results

我使用 t20 作為測資. 從量測結果可看到，SM efficiency 達 99.96%，顯示運算階段幾乎沒有 idle，running softmax、register 累積與 tiled loading 的設計有效隱藏了大部分記憶體延遲。雖然 achieved occupancy 約 0.62，但因為每個 thread 需要較多 registers 來保存 1×d 的 running state，此 occupancy 已足以維持 pipeline 高度飽和，因此不會成為瓶頸。

在記憶體使用上，shared memory load throughput 高達 599 GB/s，表示 K/V tiles 在 shared memory 中被高度重複使用；同時 global load 只有 262 GB/s、global store 只有 75 MB/s，符合 FlashAttention 不 materialize S、僅對 Q/K/V 做 tiled 讀取並最終只寫回 O 的目標。

```
==785339== Profiling result:
==785339== Metric result:
Invocations                               Metric Name                       Metric Description         Min         Max         Avg
Device "NVIDIA GeForce GTX 1080 (0)"
    Kernel: flash_attention_mixed_kernel
          1                             sm_efficiency                   Multiprocessor Activity      99.96%      99.96%      99.96%
          1                        achieved_occupancy                        Achieved Occupancy    0.624304    0.624304    0.624304
          1                    shared_load_throughput             Shared Memory Load Throughput  599.46GB/s  599.46GB/s  599.46GB/s
          1                   shared_store_throughput            Shared Memory Store Throughput  178.02GB/s  178.02GB/s  178.02GB/s
          1                           gld_throughput                  Global Load Throughput  262.23GB/s  262.23GB/s  262.23GB/s
          1                           gst_throughput                 Global Store Throughput  74.924MB/s  74.924MB/s  74.924MB/s
```

Experiment & Analysis

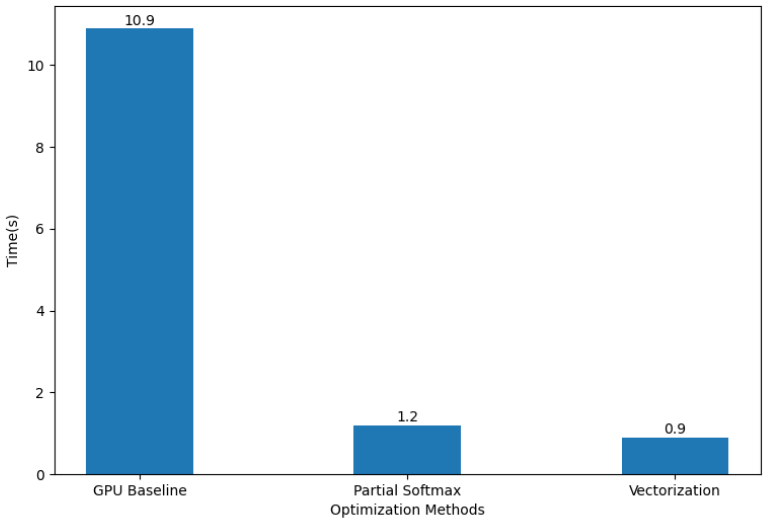
I、 System Spec

使用課堂提供的 Apollo Server

II、Optimization

實驗跑在 t19 這筆測資上。從測量的執行時間來看，三種優化對 kernel 效能的提升非常明顯。GPU Baseline 版本耗時約 10.9 秒。加入 Partial Softmax 後，耗時大幅下降到 1.2 秒，這說明將 softmax 以 online / running max 與 running sum 的方式處理，避免建立整張 N×N score matrix，有效降低了 memory 需求並提升了計算效率。再進一步加入 Vectorization（利用 float4 進行 dot product 計算），耗時進一步降低到 0.9 秒。整體來說，兩個優化步驟分別帶來約 9×

和 1.3× 的加速，最終使原始 kernel 的效能提升超過 12 倍。



AMD GPU Porting and Analysis

I、 使用 hipify-clang 產生.hip code

II、AMD 的一個 wavefront 是 64 threads，而非 NVIDIA 的 32-thread warp，因此我將每個 block 的 thread 數設為 64（Br=64），以正好對應一個 wavefront，減少硬體浪費並提升 occupancy。

Conclusion

在這次的作業中，我將課堂上學到的理論知識實際應用在深度學習的 attention 計算中。透過 partial softmax 技巧，我利用 running max 與 running sum 在每個 tile 即時計算 softmax，避免生成完整的 NxN score matrix；同時搭配 vectorization，大幅提升每個 thread 的運算效率與 GPU throughput。